Class06: R Functions Lab

Yassaman Fakhari (PID A18541145)

Table of contents

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguements* (one or more comma seperated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A First Function

Here we will create a function to add some numbers. Let's call it 'add()'. Input arguements can be either **"required"** or **"optional"**. The later have fall-back default values that will be used if the user does not specify them.

> **Q1a.** Your first version of the function should add two input numbers together. For example, add(4, 7) should return 11. [1 pt]

```
add <- function(...) {
  return(sum(...))
}
```

Can we use our function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

```
add(4, 7)
```

```
[1] 11
```

> **Q1b.** For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)). [1 pt]

```
add(4, 7)
```

```
[1] 11
```

```
add( c(4,7,10))
```

```
[1] 21
```

> **Q1c.** Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the … (dots) argument or the base R function sum(). [2 pts]

```
add(1, 2, 3, -4)
```

```
[1] 2
```

```
add(1, 2, 3)
```

```
[1] 6
```

```
ans <- add(1, 2 ,3)
ans
```

```
[1] 6
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the 'return()' function call.

```
add <- function(x, y=0, z=0) {
  return (sum(x,y,z))
  cat("Is it break time yet?")
}

add(10, 100)
```

```
[1] 110
```

Write a generate_dna() function

A useful function here is the "base R" 'sample()' function:

```
sample(1:5, size=3)
```

```
[1] 1 5 4
```

```
sample(1:5, size=6, replace=TRUE)
```

```
[1] 3 5 3 2 3 3
```

We can use this to make a random nucleotide sequence if we draw from "A", "C", "G" and "T"...

```
sample(x=c("A", "C", "G", "T"), size=10, replace=TRUE)
```

```
 [1] "C" "C" "T" "T" "G" "G" "C" "T" "G" "A"
```

Q2. Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C". [1 pt]

```
generate_dna <- function(len=10) {
  sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)
}
```

```
generate_dna(10)
```

```
 [1] "G" "A" "C" "G" "A" "T" "T" "G" "A" "G"
```

```
generate_dna(100)
```

```
  [1] "A" "C" "G" "G" "C" "A" "A" "C" "T" "G" "T" "G" "G" "G" "G" "C" "G" "A"
 [19] "T" "C" "T" "A" "A" "C" "T" "G" "C" "T" "C" "A" "T" "G" "G" "G" "T" "A"
 [37] "A" "G" "C" "T" "G" "G" "A" "C" "C" "G" "C" "C" "T" "T" "T" "A" "G" "C"
 [55] "C" "A" "G" "G" "C" "C" "A" "G" "G" "G" "T" "A" "T" "G" "T" "C" "T" "C"
 [73] "G" "C" "A" "G" "T" "G" "A" "A" "C" "T" "G" "A" "C" "C" "A" "T" "C" "A"
 [91] "T" "T" "T" "A" "C" "C" "A" "A" "A" "C"
```

Q2b. Your second version should **optionally** be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG". [1 pt]

```
generate_dna <- function(len=10,single.element=TRUE) {
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)
  if(single.element) {
    ans <- paste(ans, collapse= "")
  }
  ## Format as FASTA with an ID line
  cat(paste(">len", len, "\n", sep=""))
  cat(ans)
  cat("\n")
```

```
  ##
  ##
  return(ans)
}
```

Functions that could be helpful here are 'paste()', 'if()', 'cat()' and 'return()'.

```
generate_dna(4)
```

```
>len4
TTTT

[1] "TTTT"
```

```
generate_dna(single.element = TRUE)
```

```
>len10
ACCTGGGTGT

[1] "ACCTGGGTGT"
```

```
paste( c("A", "C", "G"), collapse="---")
```

```
[1] "A---C---G"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
cat("hello \n there")
```

```
hello
 there
```

Write a 'generate_protein()' function

Q3. Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example 'generate_protein(6)' might return "WQRTAG".

```
generate_protein <- function(len) {
  aa <- c("A","R","N","D","C","E","Q","G","H","I",
          "L","K","M","F","P","S","T","W","Y","V")

  ans <- sample(aa, size = len, replace = TRUE)

  return(paste(ans, collapse = ""))
}
```

```
generate_protein(5)
```

```
[1] "HYRNC"
```

Generate Random Protein Sequences of Length 6 to 13

> **Q4.** Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp.

```
for(l in 6:13) {
  cat(">", l, "\n", sep="")
  cat( generate_protein(l), "\n")
}
```

```
>6
PAWRVD
>7
NYEFTPS
>8
RPIESWLE
>9
NPGPWMYGC
>10
EIFAKDPNHA
>11
MNHDRTNHTFK
>12
SMALFYNIPQWY
```

```
>13
MIYLEKTLSEPHM
```

```
generate_protein(6)
```

```
[1] "WVYKVC"
```

```
generate_protein(7)
```

```
[1] "YPCIKTE"
```

```
generate_protein(8)
```

```
[1] "WMVNCFNH"
```

```
generate_protein(9)
```

```
[1] "EKHVREPCM"
```

Are Our Peptides "Unique in Nature"?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length (aa)	Ide	Cov	Unique?
6	100	100	N
7	100	100	N
8	100	100	N
9	100	100	N
10	90	100	Y
11	88	82	Y
12	90	83	Y
13	88	69	Y

Connecting Your Findings to Immunology (MHC class II and T-cell activation)

> **Q6.** In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to presentvery short peptides? [3 pt]

Looking at my Q5 results, peptides start to appear "unique in nature" at around 10 amino acids, since shorter sequences (6–9 aa) consistently showed 100% identity and coverage. This is because short peptides are very likely to already exist in known proteins, given the limited number of possible combinations. As peptide length increases, the number of possible sequences grows rapidly, making longer sequences less likely to match existing ones. Because of this, presenting very short peptides would be a poor design choice for the immune system, as they could be mistaken for "self" and fail to trigger a response. Longer peptides are more likely to be unique to pathogens, allowing better discrimination between self and non-self.